KAIRO QUESTION INTELLIGENCE MODEL (QIM)

The Architecture That Turns a Question Bank Into a Learning System

(Builds on the KAIRO Learning Engine — Phase 1: Knowledge Model, Retention States, Macro-States, Recommendation Engine — and Phase 2: Memory Scheduling, Performance Score, Kai's Behavioral Framework, Motivation, Adaptive Difficulty, Recovery. Does not redesign either.)

---

0. THE GOVERNING QUESTION

Every subsystem below exists to answer one question, continuously, per question object:

"What does this question actually know, and what can it teach the system about the student who just answered it?"

Not: is the answer right or wrong. Not: what's the next question in the bank. Not: how many questions has the student burned through today. **What is the maximum educational signal that can be extracted from this single moment of contact between a student and a question.**

If a design decision can't be traced back to that question, it doesn't belong in the model.

---

1. THE PHILOSOPHY OF A QUESTION

Most CBT practice platforms treat a question as a static object: stem, four options, one correct answer, maybe a one-line explanation. The question is inert. It does not know anything about itself, and it does not learn anything from being answered.

Inside KAIRO, a question is not an inert object. It is a **sensor**.

A single well-constructed UTME question, correctly understood, contains an enormous amount of compressed educational information — most of which traditional practice platforms discard the moment the student taps an option:

- **What concept is actually being tested** — not the topic label on a syllabus, but the exact atomic unit of understanding (the same Concept Node vocabulary from the Learning Engine).
- **What prerequisite knowledge is silently assumed** — a stoichiometry question assumes mole concept is solid; the question doesn't say this out loud, but it is testing it anyway.
- **What specific error a wrong answer reveals** — each distractor in a well-built UTME question is rarely random. It usually represents a predictable misconception, a common calculation slip, or a rule misapplied from an adjacent concept. A distractor is a diagnostic instrument, not just a wrong option.
- **How the question is likely to be encountered under real exam conditions** — reading load, time pressure, multi-step reasoning, and calculation load all shape how a student who "knows"

the concept can still get it wrong.

- **Where the question sits in a web of related questions** — easier and harder variants, same concept in different phrasing, questions that test the same misconception from a different angle.

This means every question answered by a student is simultaneously two things happening at once:

1. **A test of the student** — did they get it right, and if not, why not.
2. **A test of the question itself** — is it doing its job well (is it well-calibrated, is its concept tagging accurate, is its difficulty rating honest, is its explanation actually teaching anything).

A traditional quiz platform only runs process 1. KAIRO runs both, continuously, and the two feed each other. This is the philosophical core of the QIM: **one answered question should teach both the student and the system something valuable** — never just one of the two.

This is also the direct question-level expression of the TECHMED brand principle that content should not exist "merely because" it's technically available. A question bank of 50,000 unstructured questions is activity. A Question Intelligence Model is understanding.

---

2. QUESTION METADATA

Every question in KAIRO carries a structured metadata object. The fields below are grouped by function, and each is evaluated against three tests: **why it exists, how it helps the learning engine, and whether the student ever sees it.**

2.1 Curriculum Placement

Field	Why it exists	How it helps the engine	Student-visible?
Subject	Top-level content organisation	Session-level subject balancing (Learning Engine §4.1)	Yes
Topic	Human-readable curriculum location	Maps question to the Content Map hierarchy	Yes
Sub-topic	Finer curriculum granularity than a single Concept Node usually corresponds to	Bridges curriculum language (what a syllabus calls it) to concept language (what the engine reasons over)	Yes
Learning objective	States what mastery of this question is supposed to demonstrate, in plain language	Lets Kai explain *why* a question was chosen ("this checks whether you can apply mole ratio, not just recall it")	Indirectly, via Kai

2.2 Concept Attachment

Field	Why it exists	How it helps the engine	Student-visible?
Concept(s) tested	The real unit the engine reasons over — one question can test 1 or more Concept Nodes (Learning Engine §1.1)	This is the join key between the Question Intelligence Model and the Knowledge Model. Without it, the two systems can't talk to each other	No
Primary vs. secondary concept	Distinguishes the concept the question is *really* about from concepts it merely touches in passing	Prevents a question that lightly brushes a concept from over-updating that concept's retention state	No
Prerequisite concepts	Names what must already be solid for this question to be fairly answerable	Feeds the "check the prerequisite before assuming the advanced concept is the problem" logic (Learning Engine §1.2)	No

2.3 Difficulty & Cognitive Load

Field	Why it exists	How it helps the engine	Student-visible?
Difficulty rating	A calibrated score, not a guess (full model in Section 5)	Drives Adaptive Difficulty (Learning Engine Phase 2 §9) at the question-selection level	Sometimes — as a relative signal ("this one's a step up"), never as a raw number
Cognitive level	Where the question sits on a recall → application → analysis → synthesis spectrum (Bloom-style, adapted for UTME)	Prevents a session from being all-recall or all-application; supports the Compounding Macro-State's push toward applied/cross-concept questions	No
Estimated solving time	A calibrated, population-derived expectation of how long a well-prepared student needs	Feeds response-time baselining (Learning Engine §1.3) and exam-pressure simulation (Section 7)	Only inside timed/exam-simulation modes
Reading load	Separates comprehension difficulty from subject difficulty	Prevents misdiagnosing a `misread_question` error as a subject-knowledge gap (Learning Engine §1.4)	No
Calculation load	Flags multi-step arithmetic/algebraic demand independent of conceptual difficulty	Helps distinguish `careless_slip` (execution error under load) from `conceptual_gap`	No

2.4 Diagnostic Fields

Field	Why it exists	How it helps the engine	Student-visible?
Common misconception(s)	Names what each wrong option is likely to reveal (full framework in Section 4)	This is what makes error_pattern_tags (Learning Engine §1.4) accurate rather than guessed from response time alone	No
Skills assessed	Names the transferable skill beyond the specific concept (e.g., "unit conversion," "graph interpretation")	Lets the engine notice when a skill — not a topic — is the actual recurring weak point across subjects	No

2.5 Provenance

Field	Why it exists	How it helps the engine	Student-visible?
Question source	Where the question originated (past UTME paper, TECHMED-authored, AI-assisted, partner content)	QA accountability; lets quality issues be traced back to a source and pattern-matched	No
Year	Exam year, where applicable	Supports "how has this concept trended across recent UTME papers" curriculum-team analysis	Sometimes, in exam-simulation context
Exam body	JAMB, WAEC, NECO, institution-specific Post-UTME, etc.	Required for Future Expansion (Section 12) — the model must not assume UTME is the only exam body forever	Yes, where relevant

2.6 Relational Fields

Field	Why it exists	How it helps the engine	Student-visible?
Similar questions	Same concept, different surface wording	Supports "varied question framing" testing that produces honest confidence_score (Learning Engine §1.3)	No
Follow-up questions	A natural next question if this one is answered correctly or incorrectly	Powers the question-level interrupt logic (Learning Engine §4.2)	No
Revision priority	A derived, not authored, field — how urgently this question should resurface for a given student	This is a *per-student* runtime value, not a static property of the question — computed from the Memory Scheduling engine (Learning Engine Phase 2 §5)	No

Design note: Some fields above (revision priority, most diagnostic fields, all concept attachments) are explicitly never shown to the student. This is a deliberate expression of

"Clarity is a product feature" — clarity means the student sees what they need to make a decision, not a dump of every field the system tracks about them. Kai translates the metadata into plain language; the raw metadata itself stays internal.

---

3. CONCEPT MAPPING

3.1 The join between questions and concepts

The Question Intelligence Model does not own the Knowledge Model — the Learning Engine does (Phase 1, Section 1). The QIM's job is to be the most reliable possible *input* into it. Every question exists to update one or more Concept Nodes; a question with no concept mapping is not usable inside KAIRO, no matter how well-written it is.

3.2 Single-concept questions

The simple case: one question, one Concept Node, one clean signal. These are the backbone of the diagnostic and early-Forming-state phases (Learning Engine §3.1, Orienting Macro-State) because they produce unambiguous state updates — a wrong answer can only mean one thing.

3.3 Multi-concept questions

Most real UTME questions, especially at higher difficulty, test more than one concept simultaneously (e.g., a stoichiometry question that also requires correct mole-to-mass conversion). These are essential — they are what "applied/cross-concept questions" (Learning Engine §3.1, Compounding Macro-State) actually are structurally — but they carry a diagnostic cost: a wrong answer is ambiguous about *which* concept failed.

The QIM resolves this by weighting concept attribution (primary vs. secondary, Section 2.2) and, where the question's structure allows it, tagging which specific step of a multi-step question maps to which concept. A wrong final answer with correct working up to the last step attributes the failure narrowly, not across every concept the question touches.

3.4 Foundational vs. advanced concepts

Every concept carries a depth marker inherited from its position in the dependency graph (Learning Engine §1.2). Foundational concepts (mole concept, basic algebraic manipulation, core grammar rules) sit at the base of many dependency chains; advanced concepts sit at the top and are reached only after their prerequisites are Held.

This matters for question *selection*, not just tagging: a foundational concept should have a deep, high-quality question pool, because it is the load-bearing wall for many advanced questions downstream. A gap in the foundational pool is more damaging than a gap in an advanced one.

3.5 Dependencies between concepts, expressed through questions

Where the Learning Engine encodes dependency as an abstract graph edge, the QIM makes that edge *concrete and testable* by ensuring every prerequisite relationship has at least one question that isolates the prerequisite cleanly. Without this, the Recommendation Engine's "check the prerequisite first" logic (Learning Engine §1.2) has nothing to actually serve the student when it decides a prerequisite check is needed.

---

4. MISCONCEPTION LIBRARY

4.1 A wrong answer as a diagnostic event, not a scoring event

The Learning Engine already tags *why* a student got something wrong at the attempt level (`error_pattern_tags`, §1.4). The Misconception Library is the question-side counterpart: it is where those tags come *from*. A question's wrong-answer options are pre-analyzed, not tagged reactively after the fact.

For every distractor in a well-built question, the QIM asks: **if a student selects this option, what does that most likely mean about their thinking?**

Representative misconception categories:

- **Memorised, not understood** — the student can reproduce a fact or formula but selects an answer that shows they cannot apply it when the surface wording changes. Revealed by comparing performance across "similar questions" (Section 2.6) with different phrasing of the same concept.
- **Rushed under time pressure** — response-time pattern combined with an error type that wouldn't occur if the student had re-read the question. Distinct from not knowing the material at all.
- **Guessed** — response time and/or pattern inconsistent with genuine reasoning (Learning Engine §1.4's `guessed` tag draws directly from this).
- **Confused two similar concepts** — the wrong option corresponds exactly to the answer that *would* be correct under an adjacent, different rule. This is the clearest, most useful misconception signal a distractor can carry, because it names the exact confusion rather than just "wrong."
- **Missing prerequisite knowledge** — the error pattern is consistent with a gap further down the dependency chain, not a failure at the tested concept itself.
- **Misunderstood terminology** — the student's working (where visible) or answer pattern suggests they misread what a specific term in the question means, rather than misunderstanding the underlying science or method.

4.2 How misconceptions accumulate

A single instance of a misconception is a data point, not a conclusion. The QIM tracks misconception occurrences per student, per misconception type, across time and across concepts. When the same misconception type recurs across *different* concepts (e.g., "confuses similar terms" showing up in both Biology and Chemistry vocabulary-heavy questions), that is a stronger and more actionable signal than any single wrong answer — it points toward a transferable skill gap (Section 2.4's "skills assessed" field) rather than a subject-specific one.

This accumulated misconception profile is what allows Kai to eventually say something like "I noticed the mole concept keeps tripping this up" (Learning Engine Phase 2 §7.5) with genuine evidence behind it, rather than a generic-sounding observation.

---

5. DIFFICULTY MODEL

5.1 Why exam year is a weak proxy

Many practice platforms difficulty-rate a question by which year's paper it came from, on the unstated assumption that recent papers are harder or that difficulty is stable across years. Neither assumption holds reliably. A 2015 question can be harder than a 2023 one. Difficulty has to be assessed on the question's own structural properties, not inherited from its publication date.

5.2 What actually determines difficulty

- **Concept complexity** — how many sub-steps of reasoning the underlying concept itself requires, independent of how it's asked.
- **Reading load** — sentence complexity, length, and the presence of embedded conditions the student must track simultaneously.
- **Calculation load** — number of arithmetic/algebraic steps and how easily an early error compounds into a wrong final answer.
- **Multi-step reasoning** — whether the question requires chaining two or more concepts together (ties directly into Section 3.3's multi-concept questions).
- **Time pressure** — how the question performs specifically under timed/exam-simulation conditions versus untimed practice; some questions are only "hard" because of the clock.
- **Typical student performance (empirical calibration)** — actual population-level accuracy and response-time data on the question, once enough attempts exist. This is the ground truth that the structural factors above are ultimately calibrated against.

5.3 Difficulty adapts — it is not a fixed label

A newly imported question starts with an *estimated* difficulty, derived from the structural factors above (Section 5.2, factors 1–5). Once real attempt data accumulates, the empirical factor (5.2.6) increasingly overrides the estimate. A question authored to feel "medium" that the population consistently gets wrong should be re-rated hard — the question's real behavior in the field outranks its intended design.

This mirrors the Learning Engine's own posture toward self-reported confidence (Phase 1 §1.3): **behavior is trusted over intention.** A question doesn't get to declare its own difficulty any more than a student gets to self-declare mastery.

5.4 Difficulty is per-student-facing, not globally fixed at delivery time

The *question's* difficulty rating is global and stable (once calibrated). What varies is whether that difficulty is appropriate to surface to a *given* student right now — that decision belongs to Adaptive Difficulty (Learning Engine Phase 2 §9), which reads the question's difficulty rating as one input among several, alongside the student's own concept-level state and current Macro-State.

6. QUESTION RELATIONSHIPS

6.1 Questions as a network, not a list

A question bank organized as `Subject → Topic → flat list of questions` is a filing cabinet. The QIM instead treats questions as nodes in a relationship graph, mirroring the way concepts relate to each other (Section 3, Learning Engine §1.2).

6.2 Relationship types

Relationship	What it means	Why it matters
Prerequisite	Question B assumes mastery of what Question A tests	Enables the engine to insert a prerequisite check *before* an advanced question, using an actual question rather than an abstract concept reference (Learning Engine §4.2)
Reinforcement	Same concept, similar difficulty, different surface details	The raw material for spaced re-exposure (Phase 2 §5) and for confirming a `Reinforced` state transition is real, not a fluke
Extension	Builds one step further on the same concept	Natural next step for a student who just cleared a concept and is trending toward Compounding
Alternative wording	Same concept, deliberately different phrasing/context	The mechanism behind "varied question framing" (Learning Engine §1.3) that produces an honest confidence_score
Higher / lower difficulty	Same concept, calibrated difficulty step up/down	Powers difficulty adjustment (Section 5.4) without leaving the concept the student is currently working on
Same misconception	Different concept, same underlying error pattern	Surfaces cross-subject skill gaps (Section 4.2)
Same learning objective	Different question, same stated purpose (Section 2.1)	Useful for curriculum-team QA — confirms an objective has adequate pool depth
Bridge question	Deliberately spans two concepts to test whether a student can connect them	The concrete implementation of "cross-topic" questions referenced throughout the Learning Engine's Compounding and exam-proximity logic
Challenge question	Above the student's current calibrated level, offered as an earned stretch	Used sparingly, and only when Macro-State and difficulty math both agree the student has room (Learning Engine Phase 2 §9.2)

6.3 Why the network matters

Every one of these relationships exists so that when the Recommendation Engine decides *what kind* of next question is needed (Learning Engine §4.2 — foundational insert, reinforcement, diagnostic, etc.), the QIM can hand back an actual, specific, appropriately-calibrated question instead of the engine having to search the entire bank on the fly. The relationship graph is what makes the Recommendation Engine's real-time interrupt logic *fast* as well as *correct*.

7. ADAPTIVE RECOMMENDATIONS

Question metadata is only valuable if it actually changes what the student sees. This section maps QIM fields to the moments where they're consumed.

- **Today's Mission** — pulls from urgent-decay concepts (Phase 2 §5) and translates them into actual question selections using the relationship graph (Section 6) and current difficulty calibration (Section 5).
- **Practice sessions** — session-level queue building (Learning Engine §4.1) draws directly on concept mapping (Section 3) and difficulty (Section 5) to hit the five-input priority order without manual curation.
- **Revision** — resurfaces Fading concepts using Reinforcement and Alternative-wording relationships (Section 6.2), never the identical question the student saw the first time, which would test memorization of the question rather than the concept.
- **Recovery sessions** — deliberately pulls from lower-difficulty, high-confidence-concept questions (Learning Engine §10.2's "reconnection session"), using the Lower-difficulty relationship type to stay on the *same* concept at reduced difficulty rather than switching topics.
- **Confidence rebuilding** — favors questions with strong empirical calibration (Section 5.3) and Held-state concepts, so early wins are genuine, not artificially easy.
- **Challenge mode** — draws exclusively from Extension and Challenge-question relationships, gated by Macro-State (Compounding/Peak Readiness only, per Learning Engine Phase 2 §9.2).
- **Exam simulations** — weights Estimated solving time, Reading load, and Calculation load (Section 2.3) heavily, because the goal shifts from concept acquisition to reproducing real exam-day conditions (Learning Engine §5.2's exam-proximity override).
- **Kai's explanations** — draws on Learning objective, Common misconception, and Skills assessed (Sections 2.1, 2.4) to generate the plain-language "why this question" framing described in Learning Engine Phase 2 §7.5.

---

8. LEARNING AFTER EVERY ATTEMPT

When a student answers one question, the QIM contributes the following updates — feeding directly into the Knowledge Model and Motivation systems already defined in the Learning Engine:

- **Concept mastery changes** — the attempt updates `retention_state` and `decay_estimate` on every attached Concept Node (Section 2.2), weighted by primary/secondary attribution.
- **Confidence changes** — feeds into `confidence_score` (Learning Engine §1.3), using this question's calibrated difficulty and cognitive level as context — a correct answer on a hard, multi-step question moves confidence more than a correct answer on an easy recall question.
- **Memory changes** — if the concept was in a Fading state, a correct answer is evaluated as a candidate `Reinforced` transition (Learning Engine §2.2); if in Held, it either quietly confirms the state or, if wrong, triggers the recalibration described in Learning Engine Phase 2 §5.4.

- **Speed trend** — the attempt's response time is compared against both the question's population-calibrated Estimated solving time (Section 2.3) and the student's own personal baseline (Learning Engine §1.3), updating both independently.
- **Guess detection** — response time, pattern, and the question's own misconception profile (Section 4) are cross-checked; a suspiciously fast correct answer on a previously Fading concept is treated with lower confidence rather than immediate promotion (Phase 2's "gaming the system" edge case, §11).
- **Misconception updates** — if wrong, the selected distractor is matched against the question's pre-analyzed misconception profile (Section 4.1) to produce the `error_pattern_tag` the Learning Engine consumes directly.
- **Readiness updates** — contributes to the Elite Score's Accuracy and Retention components (Learning Engine Phase 2 §6.2), weighted by this question's difficulty and recency.
- **Learning momentum** — contributes one data point to the Consistency component (§6.2) and, in aggregate across a session, to Macro-State transition signals (Learning Engine §3.1).

Every one of these updates happens from a single attempt. This is the mechanical answer to the philosophy stated in Section 1: **one answered question teaches the system multiple things at once, not just "correct" or "incorrect."**

---

9. EXPLANATION INTELLIGENCE

9.1 An explanation is a teaching moment, not a correction notice

A weak explanation says "the answer is B." A QIM-driven explanation uses the question's own metadata to teach through the specific moment the student is in.

9.2 Structure

- **Correct reasoning** — the shortest accurate path to the right answer, using the question's tagged concept(s) (Section 2.2) as the explanation's spine.
- **Why each wrong option is wrong** — drawn directly from the Misconception Library (Section 4); each distractor's explanation names the specific reasoning error it represents, not just "this is incorrect."
- **Common mistake** — surfaced only when it's genuinely common (backed by empirical distractor-selection data across the student population), framed neutrally, never as "many people are foolish enough to think..."
- **Exam tip** — a transferable strategy tied to the question's Skills assessed field (Section 2.4), useful beyond this specific question.
- **Memory anchor** — a short, concrete hook designed to aid recall at the next Fading-state encounter — this is explanation content built with Section 8's future retrieval in mind, not just this moment.
- **Related concept** — a pointer into the dependency graph (Section 3.5), useful when the student's error pattern suggests the real gap is one level down.
- **Suggested follow-up question** — pulled directly from the question's relationship graph (Section 6.2 — Reinforcement or Extension type), not a random next item.

- **Kai's encouragement** — generated per the tone rules already defined in the Learning Engine (Phase 2 §7.2, §7.3) — specific and evidence-based, never generic praise.

9.3 Length is adaptive, not fixed

- **Shorter explanations** are appropriate when the error pattern is `careless_slip` (Learning Engine §7.3's table — light touch, no re-explanation needed) or when the concept is already Held and this was a rare miss.
- **More detailed explanations** are appropriate for `conceptual_gap` on a Forming concept, or when a student's Macro-State is Orienting and the system should over-explain rather than under-explain (Learning Engine §7.4).

This directly implements Learning Engine Phase 2 §7.3's response library at the content-generation layer — the QIM produces the raw material; Kai's behavioral framework decides which pieces to use and how much of each.

---

10. QUESTION LIFECYCLE

Every question that enters KAIRO moves through a defined lifecycle:

```
Imported
  ↓
Reviewed (curriculum team checks accuracy, clarity, fairness)
  ↓
Tagged (metadata fields from Section 2 populated)
  ↓
Linked to concepts (Section 3 — join to the Knowledge Model)
  ↓
Assigned misconceptions (Section 4 — each distractor analyzed)
  ↓
Used in practice (enters live rotation, Section 7)
  ↓
Updates student model (Section 8, on every attempt)
  ↓
Improves recommendations (feeds Learning Engine §4.1/§4.2 in real time)
  ↓
Appears in review (Memory Scheduling, Learning Engine Phase 2 §5)
  ↓
Supports weekly report (Weekly Reflection, Learning Engine Phase 2 §8.2)
  ↓
Contributes to exam readiness (Peak Readiness Macro-State, exam simulations)
```

A question does not stop changing once it's "used." Its difficulty rating (Section 5.3), its relationship links (Section 6), and even its misconception profile (Section 4.1) can all be refined as real attempt data accumulates — the lifecycle is a loop that feeds back into earlier stages, not a one-way pipeline.

---

11. QUALITY ASSURANCE

No question enters live rotation without satisfying the following standards:

- **Educational quality** — does the question actually test meaningful understanding, not trivia or ambiguous trick-wording.
- **Accuracy** — the marked correct answer is verified correct; a wrong answer key silently corrupts every downstream system that trusts it.
- **Clarity** — the stem is unambiguous; a poorly worded question produces `misread_question` errors that aren't really about the student.
- **Fairness** — the question doesn't assume knowledge outside its stated concept/prerequisite scope, and doesn't rely on cultural or contextual assumptions that disadvantage some students unfairly.
- **Difficulty validation** — the initial estimated difficulty (Section 5.2) is sanity-checked against similar existing questions before the empirical calibration (5.3) has enough data to take over.
- **Duplicate detection** — checked against the Similar-questions and Alternative-wording relationship pools (Section 6.2) to avoid silently inflating a concept's apparent pool depth with near-identical items.
- **Metadata completeness** — every required field from Section 2 is populated before a question can be tagged "ready for rotation"; an incompletely tagged question is a silent hole in the Knowledge Model.
- **Explanation quality** — every explanation satisfies the Section 9 structure at minimum viable depth (correct reasoning + why each wrong option is wrong, at minimum) before the question can go live.
- **Alignment with UTME syllabus** — confirmed against the current official syllabus, since syllabus scope and emphasis do shift over time and stale alignment silently misleads students about what actually matters.

This QA gate is the direct question-level expression of "Build reliability before complexity" (TECHMED 2027 operating document, Primary Goal 3) — a wrongly-tagged or wrongly-keyed question doesn't just fail once; it corrupts every future recommendation the engine makes using it.

---

12. FUTURE EXPANSION

The QIM is architected so the following can be added without re-architecting the core model:

- **AI-generated questions** — can enter at the "Imported" lifecycle stage (Section 10) and pass through the identical Review → Tag → Link → QA pipeline as human-authored ones; the model does not need to know or care how a question was originally produced, only that it passes the same gate.
- **Question improvements** — because difficulty (Section 5.3), relationships (Section 6), and misconception profiles (Section 4) are all designed to update from real data rather than stay fixed at authoring time, "improving" a question is a native, expected lifecycle event, not a special case.
- **Multiple exam bodies** — the Exam body field (Section 2.5) and the subject-agnostic Content Map / Knowledge Map separation (Learning Engine §1.1) mean extending to WAEC, NECO, or

institution-specific Post-UTME papers is a matter of tagging and content, not new architecture.

- **Post-UTME, university-level learning** — the same Concept Node + dependency graph structure scales upward in the same way the Learning Engine's own Section 12 already anticipates.
- **Interactive, image-based, and diagram questions** — the metadata model (Section 2) is format-agnostic; Reading load and Calculation load (Section 2.3) generalize naturally to "visual interpretation load" for diagram-based items without new top-level fields.
- **Video and voice explanations** — Section 9's explanation structure is medium-agnostic; the nine content components (correct reasoning, misconception breakdown, exam tip, etc.) are script content that can be rendered as text, voice, or video without changing what's actually being taught.

FINAL OUTPUT — WHY THE QIM IS KAIRO'S BRAIN

The Learning Engine knows *what a student knows*. The Student Intelligence Model knows *who the student is as a learner over time*. Neither of those systems can function on good instinct alone — they need a constant, reliable stream of structured signal to reason over. That signal comes from exactly one place: the questions themselves.

A traditional question bank is a filing cabinet. It stores content and waits to be opened. The Question Intelligence Model turns every question into an active participant in the student's learning journey — something that knows what it's testing, what a wrong answer on it usually means, what should come before it and after it, and how urgently it should resurface for a specific student on a specific day.

This is what makes the difference between a platform with 50,000 questions and a platform with 50,000 questions *organized into a system that understands the student through them*. Without the QIM, Kai has nothing honest to say — no explanation grounded in a real misconception, no "I put this one in front of you because..." grounded in an actual reason, no revision priority grounded in anything but a guess. With it, every single tap a student makes on an answer option becomes a small, structured act of the system getting to know them better — which is the entire promise of Kairo: **the system should remember the student and help the student know what to do next.**

The Question Intelligence Model is not a feature of KAIRO. It is the nervous system that lets every other feature actually be intelligent.